Unité 9 — Cahier de laboratoire (étudiant)

Livrable C — Bonnes pratiques & SAST · UE « Software Security »

Code UE 7532 14 U32 D1 · 13 octobre 2026

Cadre. Environnement Docker isolé : Bandit 1.9.4, Cppcheck 2.13.0, Semgrep 1.180.0. Toutes les commandes ci-dessous sont exécutées depuis /workspace.

Lab 9.1 — SAST d’une application Python (Bandit) | EXERCICE GUIDÉ

1. Objectifs

Exécuter une analyse statique, lire et trier les résultats, comparer avec une version corrigée.

2. Prérequis

Cours U9 ; unités U7 et U8.

3. Environnement

Conteneur U09 ; fichiers src/web/app.py et src/web/app_fixed.py.

4. Fichiers

Applications Python vulnérable et corrigée, reprises des unités précédentes.

5. Commandes

```
cd /workspace
bandit src/web/app.py
bandit src/web/app_fixed.py
```

6. Manipulations

a. Relevez les règles B602 et B608, leurs sévérités et niveaux de confiance.
b. Reliez les alertes aux portions de code et aux vulnérabilités des unités U7/U8.
c. Comparez avec app_fixed.py : les alertes Medium/High doivent disparaître.

7. Résultats attendus

B602 : High / High (shell=True) ; B608 : Medium / Low (SQL construit dynamiquement). Sur app_fixed.py, seules des alertes Low peuvent subsister.

8. Explications

Un SAST signale des motifs suspects ; la revue du code permet de déterminer leur pertinence.

9. Questions

1. Pourquoi shell=True est-il classé High ?
2. Que signifie la confiance Low du résultat SQL ?
3. Un SAST garantit-il l’absence de vulnérabilités ?

10. Corrigé

Livrable D, Lab 9.1.

11. Vulnérabilité et remédiation

CWE-78 / CWE-89 : éviter shell=True et utiliser des requêtes SQL paramétrées.

12. Durée

1 h.

Lab 9.2 — SAST d’un code C (Cppcheck) | EXERCICE GUIDÉ

1. Objectifs

Détecter des erreurs mémoire par analyse statique.

2. Prérequis

Lab 9.1 ; unité U3.

3. Environnement

Conteneur U09 avec Cppcheck.

4. Fichiers

src/heap/double_free.c, uaf.c, heap_of.c et uninit.c.

5. Commandes

```
cd /workspace
cppcheck --enable=warning,style \
  src/heap/double_free.c src/heap/uaf.c \
  src/heap/heap_of.c src/heap/uninit.c
```

6. Manipulations

a. Lancez Cppcheck sur les quatre fichiers.
b. Localisez doubleFree et bufferAccessOutOfBounds.
c. Comparez les bugs détectés et non détectés avec ASan/Valgrind (U3/U5).

7. Résultats attendus

doubleFree dans double_free.c (ligne 8) et bufferAccessOutOfBounds dans heap_of.c (ligne 7). L’absence d’alerte pour uaf.c ou uninit.c ne prouve pas l’absence de bug.

8. Explications

L’analyse statique ne détecte pas systématiquement les erreurs dépendantes de l’exécution.

9. Questions

1. Quelles lignes sont signalées ?
2. Quels bugs restent non détectés ?
3. Pourquoi combiner analyse statique et dynamique ?

10. Corrigé

Livrable D, Lab 9.2.

11. Vulnérabilité et remédiation

CWE-415 et CWE-787 : gérer la durée de vie des pointeurs et respecter les limites des tampons.

12. Durée

45 min.

Lab 9.3 — Audit combiné | EXERCICE AUTONOME

1. Objectifs

Auditer un code fourni : SAST, revue manuelle, confirmation dynamique et rapport.

2. Prérequis

Labs 9.1 et 9.2 ; unités U5, U7 et U8.

3. Environnement

Conteneur U09 pour le SAST ; environnements des unités précédentes pour les tests dynamiques.

4. Fichiers

src/web/app.py et, au choix de l'enseignant, les exemples src/heap/.

5. Commandes

```
cd /workspace
bandit src/web/app.py
semgrep scan --config rules/u09-python.yaml src/web/app.py
# Option C : cppcheck --enable=warning,style src/heap/*.c
```

6. Manipulations

a. Lancez les SAST appropriés et confrontez leurs résultats au code.
b. Classez les résultats (vrais positifs, faux positifs possibles, limites de détection).
c. Confirmez au moins une faille avec un test déjà étudié dans U5, U7 ou U8, dans le laboratoire correspondant, et non dans le conteneur SAST isolé.
d. Proposez une correction et rédigez un rapport : constat, preuve, CWE, gravité (U10), correctif et vérification.

7. Résultats attendus

Bandit signale B602/B608 ; les deux règles Semgrep locales signalent SQLi et shell=True. Rapport structuré avec au moins une faille confirmée et un correctif justifié.

8. Explications

Un audit combine les résultats d'outils, la lecture du code et des preuves d'exécution.

9. Questions

1. Quels résultats sont des vrais positifs ?
2. Quelle faille avez-vous confirmée dynamiquement, et dans quel environnement ?
3. Comment justifier sa priorité ?

10. Corrigé

Livrable D, Lab 9.3.

11. Vulnérabilité et remédiation

Selon le code audité ; argumenter chaque correction.

12. Durée

1 h 30.

Note : les règles Semgrep sont locales (rules/u09-python.yaml). Ne pas utiliser --config auto dans ce conteneur sans réseau.